

JusticeFlow

Police Case & Evidence Management System

INTEGRATION CONTRACT PROPOSAL

Pre-Implementation Team Agreement | Spring 2026

Field	Detail
Project	JusticeFlow — Multi-Course Integrated System
Courses	DBMS · SDA · AI · OS
Batch	BCS-4D FAST-NU Karachi
Team Lead & Author	Muhammad Furqan Sheikh (24K-0527)
Teammate 2	Muhammad Abu Bakar (24K-0826)
Teammate 3	Abdullah Muaz Abbasi (24K-0764)
Document Version	v1.0 — Proposal (Open for Team Review)
Implementation Start	April 15, 2026
Status	PENDING TEAM SIGN-OFF

This document is a proposal by the Team Lead. All team members are expected to review, raise objections, propose amendments, and sign off before implementation begins. Nothing in this document is final until all three members agree.

1. Purpose & Scope

This document defines the binding integration contracts that all three team members must agree on before writing a single line of implementation code. These contracts exist for one reason: to prevent integration hell in Week 5.

Without these agreements, three separately built subsystems will meet for the first time in the final week and fail to connect — not because of bad code, but because of silent assumptions made independently. This document eliminates those assumptions.

Rule: Any change to a contract after sign-off requires explicit agreement from all three members and an updated version of this document.

1.1 What This Document Covers

- Shared data structures — C++ structs mirroring the database schema
- Authentication interface — what every subsystem calls and receives from Module 0
- Error handling convention — one approach, enforced across all subsystems
- Database conventions — types, formats, naming rules
- IPC message formats — struct definitions for inter-process communication
- PostgreSQL role usage — which role connects in which context
- Repository structure — directory layout and file ownership
- Git conventions — branching, commit format, merge rules
- Cross-subsystem guarantees — what each team member promises to deliver and when
- Demo scenario — the end-to-end flow all three subsystems must support

1.2 What This Document Does Not Cover

- Internal module implementation — each member owns their subsystem internals
- Database schema — already finalized in 02_tables.sql
- Algorithm logic — internal to each AI agent
- UI/dashboard design — decided in Week 5 integration phase

2. Shared Data Structures (common.h)

All shared C++ structs live in a single file: common/common.h. This file is owned by the Team Lead (Furqan) and is the first file pushed to the repository. No teammate modifies it without team consensus.

PostgreSQL type mapping used throughout: SERIAL/INTEGER → int | VARCHAR/TEXT → std::string | BOOLEAN → bool | TIMESTAMPTZ → time_t | NUMERIC → double | JSONB → std::string (serialized) | Custom enum → C++ enum class from constants.h

2.1 Core Entity Structs

Officer

```
struct Officer {
    int            officer_id;
    std::string    cnic;           // format: XXXXX-XXXXXX-X
    std::string    belt_number;
    std::string    first_name;
    std::string    last_name;
    std::string    rank;           // matches OfficerRank enum
    int            station_id;
    std::string    status;         // matches OfficerStatus enum
    int            active_case_count;
};
```

SessionContext

```
struct SessionContext {
    int            officer_id;
    std::string    cnic;
    std::string    rank;
    int            station_id;
    std::string    belt_number;
    time_t         created_at;
    time_t         expires_at;
    bool           is_valid;
    std::string    session_token; // UUID string
};
```

SessionContext is the single most important struct. Every subsystem receives one from Module 0. Its fields must never be renamed or retyped after sign-off.

Case

```
struct Case {
    int            case_id;
    std::string    case_number;    // format: FIR-YYYY-NNNN
    std::string    status;         // matches CaseStatus enum
    std::string    crime_type;
};
```

```

    int         station_id;
    int         filed_by_officer_id;
    double      incident_lat;    // ALWAYS populated on FIR registration
    double      incident_lon;    // ALWAYS populated on FIR registration
    time_t      filed_date;
    std::string severity;        // LOW | MEDIUM | HIGH | CRITICAL
};

```

Evidence

```

struct Evidence {
    int         evidence_id;
    int         case_id;
    std::string evidence_type;
    std::string evidence_status; // matches EvidenceStatus enum
    std::string file_path;
    bool        is_deleted;      // soft delete – never physical DELETE
    time_t      collected_at;
    int         collected_by_officer_id;
};

```

Warrant

```

struct Warrant {
    int         warrant_id;
    int         case_id;
    std::string warrant_type;
    std::string status;          // matches WarrantStatus enum
    int         issued_by_officer_id;
    time_t      issue_date;
    time_t      valid_until;
};

```

Arrest

```

struct Arrest {
    int         arrest_id;
    int         case_id;
    std::string accused_cnic;
    std::string custody_status;  // matches CustodyStatus enum
    int         arresting_officer_id;
    int         warrant_id;      // -1 if no warrant
    time_t      arrest_date;
    bool        is_disputed;
};

```

BailRecord

```

struct BailRecord {
    int         bail_id;
    int         arrest_id;
};

```

```
std::string bail_type;  
double      bail_amount;  
std::string status;           // matches BailStatus enum  
time_t      bail_date;  
time_t      valid_until;      // -1 if no expiry  
};
```

3. Constants & Enums (constants.h)

All status strings from 01_types.sql are mirrored as C++ enum classes. No teammate uses raw string literals for status values — always use the enum. This prevents typos like "ACTIVE" or "Issued" causing silent failures.

Officer Enums

```
enum class OfficerRank {
    CONSTABLE, ASI, SI, INSPECTOR, DSP, SP, SSP, DIG, IG
};

enum class OfficerStatus {
    ACTIVE, SUSPENDED, RETIRED, TERMINATED
};
```

Case & Evidence Enums

```
enum class CaseStatus {
    REGISTERED, UNDER_INVESTIGATION, CHALLAN_SUBMITTED,
    CLOSED, ARCHIVED
};

enum class EvidenceStatus {
    COLLECTED, SENT_TO_LAB, RETURNED_FROM_LAB, DESTROYED
};
```

Legal Enums

```
enum class WarrantStatus { ISSUED, EXECUTED, CANCELLED, EXPIRED };

enum class CustodyStatus {
    IN_CUSTODY, BAIL_GRANTED, REMANDED, RELEASED, ESCAPED
};

enum class BailStatus { ACTIVE, REVOKED, EXPIRED, CANCELLED };
```

Result Codes

```
enum class JFResult {
    OK = 0,
    AUTH_FAILED = 1,
    RANK_INSUFFICIENT = 2,
    NOT_FOUND = 3,
    DB_ERROR = 4,
    INVALID_STATE = 5,
    DUTY_INACTIVE = 6,
    SESSION_EXPIRED = 7,
    INVALID_INPUT = 8
};
```

```
};
```

String conversion helpers — each enum must have a `toString()` and `fromString()` helper defined in `constants.h`. This is Furqan's responsibility as `common.h` owner.

4. Authentication Interface Contract

Module 0 (Auth & Session) is shared infrastructure. Every subsystem calls it. The interface below is fixed. No subsystem reimplements its own authentication logic.

4.1 AuthManager Interface

```
class AuthManager {
public:
    // Singleton – one instance per process
    static AuthManager& getInstance();

    // Login – returns valid session or is_valid=false
    SessionContext login(const std::string& cnic,
                        const std::string& password);

    // Rank check – call before any privileged operation
    bool validateRank(int officer_id,
                    OfficerRank minimum_rank);

    // Duty check – call before case assignment
    bool isDutyActive(int officer_id);

    // Logout – closes DB connection, invalidates token
    void logout(SessionContext& session);

    // Session refresh – call on timeout
    JFResult refreshSession(SessionContext& session);
};
```

4.2 How Teammates Use Auth

Every operation that requires a logged-in officer follows this pattern:

```
// Step 1: Check session is valid
if (!session.is_valid) return JFResult::AUTH_FAILED;

// Step 2: Check rank if operation requires it
if (!auth.validateRank(session.officer_id, OfficerRank::INSPECTOR))
    return JFResult::RANK_INSUFFICIENT;

// Step 3: Proceed with operation
```

No subsystem bypasses this pattern. No hardcoded officer_ids in tests. Use the data generator accounts for testing.

4.3 Availability Guarantee

Deliverable	Deadline	Owner
AuthManager compilable and linkable (stub)	April 15 — Day 1	Furqan (24K-0527)
login() returning valid SessionContext	April 17 — Day 3	Furqan (24K-0527)
validateRank() working correctly	April 17 — Day 3	Furqan (24K-0527)
isDutyActive() working correctly	April 18 — Day 4	Furqan (24K-0527)
Full session lifecycle tested	April 21 — End Week 1	Furqan (24K-0527)

5. Error Handling Convention

One convention. Enforced everywhere. No exceptions (pun intended).

DECISION: Return codes using JFResult enum. No C++ exceptions anywhere in the codebase. Rationale: exceptions interact poorly with POSIX fork/thread boundaries and complicate the OS layer significantly.

5.1 Rules

- Every function that can fail returns JFResult
- Output is passed by reference parameter, not return value
- Caller always checks JFResult before using output
- No silent failures — every error is logged
- No raw integer error codes — always JFResult enum

5.2 Function Signature Pattern

```
// CORRECT pattern
JFResult getCase(int case_id, Case& out_case);
JFResult recordArrest(const Arrest& arrest, int& out_arrest_id);

// WRONG — do not return entity directly
Case getCase(int case_id); // BAD — how do you signal not found?
```

5.3 Logging Convention

```
// Every error must log before returning
if (result != JFResult::OK) {
    Logger::error("[S3][Warrant] Failed: " + toString(result));
    return result;
}
```

Logger is a shared utility in common/logger.h — simple file logger, thread-safe, owned by Furqan.

6. Database Conventions

These rules apply to all C++ code that interacts with PostgreSQL across all three subsystems.

6.1 Type Conventions

Field	C++ Type	PostgreSQL Type	Format / Notes
officer_id	int	SERIAL / INTEGER	Always int, never string
case_id	int	SERIAL / INTEGER	Always int, never string
evidence_id	int	SERIAL / INTEGER	Always int, never string
cnic	std::string	VARCHAR(15)	XXXXXX-XXXXXXX-X always
station_id	int	INTEGER	Always int
Timestamps	time_t	TIMESTAMPTZ	Unix epoch in C++
Latitude/Longitude	double	NUMERIC(10,7)	Always double
Status fields	enum class	Custom PG enum	Use constants.h enums
JSONB fields	std::string	JSONB	Serialize/deserialize as JSON string
Soft delete flag	bool	BOOLEAN	is_deleted — never physical DELETE on Evidence

6.2 Connection Rules

Context	PostgreSQL Role	Permissions
C++ application layer	justice_app	SELECT/INSERT/UPDATE/DELETE on public schema
Python AI agents	justice_ai	SELECT on public, INSERT on analytics only
Audit schema writes	Trigger only (SECURITY DEFINER)	No direct application writes ever
DB_ADMIN	Never used by application	Internal PostgreSQL administration only

Connection strings live in common/db_config.h only. No teammate hardcodes credentials anywhere else in the codebase.

6.3 Soft Delete Rule

Evidence is NEVER physically deleted. Always set `is_deleted = TRUE`. This is enforced by a database trigger but must also be respected in all C++ code. Any DELETE statement targeting the Evidence table in application code is a contract violation.

7. IPC Message Formats

The OS layer defines the message formats for all inter-process communication. These structs are used by the scheduler, AI agents, and shared memory segment. Defined in `common/ipc_types.h`.

7.1 Named Pipe — AI Agent Status

```
// Written by each AI agent to its named FIFO on completion
struct AgentStatusMessage {
    char    agent_name[32];          // "hotspot" | "priority" | "workload"
    time_t  completed_at;
    int     predictions_generated;
    int     error_code;              // 0 = success
    char    error_detail[128];      // empty string if no error
};
```

7.2 Shared Memory — Live Agent Status Table

```
// One entry per AI agent in shared memory segment
struct AgentStatus {
    char    agent_name[32];
    time_t  last_run_at;
    time_t  next_run_at;
    int     predictions_generated;
    double  model_accuracy;
    bool    is_running;
    int     last_error_code;
};

// Full shared memory segment layout
struct SharedStatusTable {
    pthread_mutex_t mutex;          // protects entire struct
    int           active_sessions;
    AgentStatus   agents[3];       // [0]=hotspot [1]=priority [2]=workload
};
```

7.3 FIFO Paths

```
// Defined in common/db_config.h
const char* HOTSPOT_FIFO    = "/tmp/jf_hotspot.fifo";
const char* PRIORITY_FIFO  = "/tmp/jf_priority.fifo";
const char* WORKLOAD_FIFO  = "/tmp/jf_workload.fifo";
const char* SHM_NAME       = "/jf_status_shm";
```

8. Repository Structure

The GitHub repository must follow this structure from Day 1. No teammate creates directories outside this layout without team consensus.

```
justiceflow/
├── common/
│   ├── common.h           ← shared structs (owner: Furqan)
│   ├── constants.h        ← enums, JFResult (owner: Furqan)
│   ├── db_config.h        ← connection strings, FIFO paths (owner: Furqan)
│   ├── ipc_types.h        ← IPC message structs (owner: Furqan)
│   └── logger.h           ← thread-safe logger (owner: Furqan)
├── subsystem1/            ← Officers, Personnel (owner: Abdullah)
├── subsystem2/            ← FIR, Evidence, Cases (owner: Abu Bakar)
├── subsystem3/            ← Security, Audit, Legal (owner: Furqan)
├── os_layer/              ← daemon, threads, IPC, mmap (all three)
│   ├── scheduler/         ← Furqan
│   ├── ipc/               ← Abdullah
│   └── threading/         ← Abu Bakar
├── ai_agents/             ← Python scripts
│   ├── hotspot_agent.py   ← Abdullah
│   ├── priority_agent.py  ← Abu Bakar
│   └── workload_agent.py  ← Furqan
├── dashboard/             ← Flask + Chart.js (Week 5, all three)
├── db/                    ← SQL files (already done)
├── tests/                 ← 57 test cases
├── CONTRACTS.md           ← this document summarized
├── README.md
└── Makefile               ← unified build (owner: Furqan)
```

The Makefile at root builds all three subsystems and the OS layer together. Individual subsystem builds are allowed for development but the root Makefile is the authoritative build.

9. Git Conventions

9.1 Branch Rules

Branch	Rule	Who Merges
main	Always compiles. Always runnable. No direct pushes.	Any member after review
s1/feature-name	Subsystem 1 features (Abdullah)	Abdullah after self-review
s2/feature-name	Subsystem 2 features (Abu Bakar)	Abu Bakar after self-review
s3/feature-name	Subsystem 3 features (Furqan)	Furqan after self-review
os/feature-name	OS layer features (any member)	One other member reviews
hotfix/description	Emergency fixes to main	Team Lead (Furqan)

9.2 Commit Message Format

[PREFIX] Short description of what changed

Prefixes: [S1] [S2] [S3] [OS] [AI] [DB] [COMMON] [FIX] [TEST]

Examples:

[S3] Add warrant state machine with ISSUED->EXECUTED transition

[OS] Implement double-fork daemon initialization

[COMMON] Add OfficerRank enum to constants.h

[FIX] Resolve semaphore deadlock in session handler

9.3 Rules

- Never push broken code to main — test locally first
- One teammate reviews before any OS layer merge — too many shared dependencies
- common/ files require explicit acknowledgment from all three before merge
- Commit messages in English, present tense, imperative mood
- No commented-out code in commits to main

10. Cross-Subsystem Guarantees

Each team member makes binding promises about what their subsystem delivers and when. These are the minimum requirements for the integration phase to succeed.

10.1 Furqan's Guarantees (Subsystem 3 + Shared Infrastructure)

Guarantee	Deadline
common.h, constants.h, db_config.h pushed to main	April 15 — Day 0
AuthManager stub — compilable and linkable by teammates	April 15 — Day 1
login() returning valid SessionContext	April 17 — Day 3
validateRank() and isDutyActive() working	April 17 — Day 3
Daemon initialized and logging heartbeat	April 21 — End Week 1
Unix domain socket to PostgreSQL established	April 21 — End Week 1
Officer.status always reflects actual duty state via Auth layer	April 21 — End Week 1

10.2 Abu Bakar's Guarantees (Subsystem 2)

Guarantee	Deadline
Cases.incident_lat and incident_lon always populated on FIR registration	TBD by Abu Bakar
case_id is always int — no string representations returned	TBD by Abu Bakar
Evidence soft delete always respected — is_deleted flag only	TBD by Abu Bakar
Case status transitions follow defined CaseStatus enum values only	TBD by Abu Bakar
FIR registration notifies OS scheduler via shared memory on completion	TBD by Abu Bakar

10.3 Abdullah's Guarantees (Subsystem 1)

Guarantee	Deadline
Officers.status always reflects actual duty state — no stale reads	TBD by Abdullah
officer_id is always int — no string representations returned	TBD by Abdullah
active_case_count always accurate — updated on every case assignment	TBD by Abdullah
Officer rank always uses OfficerRank enum values	TBD by Abdullah
station_id always populated — never null or -1 for active officers	TBD by Abdullah

Deadlines marked 'TBD by [member]' are to be filled in by that team member during the contract review meeting. Team Lead's deadlines are pre-committed.

11. Demo Scenario Contract

All three subsystems must be built to support this exact demo flow. It is the end-to-end scenario that will be shown to all four course instructors. Every module touched in this flow must be working before the demo.

11.1 The FIR Filing Flow

Step	Layer	What Happens	Owned By
1	OS	justice_authd authenticates officer UID and enforces rank	Furqan
2	S3	AuthManager.login() validates CNIC, returns SessionContext	Furqan
3	S2	FIR registered — INSERT into Cases, incident_lat/lon populated	Abu Bakar
4	DB	Audit trigger fires — change logged to audit.Audit_Log automatically	Furqan (trigger)
5	S2	Evidence collected and linked to case	Abu Bakar
6	OS	Scheduler spawns hotspot_agent.py — DBSCAN re-clusters	Furqan + Abdullah
7	AI	Random Forest scores new case — priority written to analytics schema	Abu Bakar
8	AI	Hungarian Algorithm re-runs — optimal officer assignment suggested	Furqan
9	S3	Warrant issued by Inspector — state machine ISSUED	Furqan
10	S3	Arrest recorded — custody state IN_CUSTODY	Furqan
11	Dashboard	SHO sees: FIR number, crime zone, priority score, assignment	All three

If any step in this flow fails during the demo, the entire integration story breaks. Every team member must test their modules against this exact flow before Week 5.

12. Sign-Off

This contract becomes binding when all three team members sign off. Sign-off means:

- I have read this document completely
- I agree with all conventions, structures, and interfaces defined
- I have raised all objections and they have been resolved
- I commit to delivering my guarantees by the agreed deadlines
- I will not unilaterally change any contract item without team agreement

Member	Roll Number	Role	Signature / Git Commit	Date
Muhammad Furqan Sheikh	24K-0527	Team Lead & Author	_____	_____
Muhammad Abu Bakar	24K-0826	Subsystem 2 Owner	_____	_____
Abdullah Muaz Abbasi	24K-0764	Subsystem 1 Owner	_____	_____

Digital sign-off: Each member pushes a commit to main that adds their name to CONTRACTS.md under the sign-off section. The commit timestamp is the binding date.

Document Author: Muhammad Furqan Sheikh (24K-0527)

Version: 1.0 — Open for Team Review

Next Version: 2.0 — Final (after sign-off)